

Program:-1(A)

//1603. Design Parking System

```
class ParkingSystem {

    int big; int medium;

    int small;

    public ParkingSystem(int big, int medium, int small) {
        this.big = big;
        this.medium = medium;
        this.small = small;
    }

    public boolean addCar(int carType) {
        if(carType == 1){
            if(big > 0){
                big--; return true;
            }
        }

        else if(carType == 2){
            if(medium > 0){
                medium--;
                return true;
            }
        }
    }
}
```

```
    else{  
        if(small > 0){  
            small--;  
            return true;  
        }  
    }  
    return false;  
}  
}
```

```
/**
```

```
 * Your ParkingSystem object will be instantiated and called as such:
```

```
 * ParkingSystem obj = new ParkingSystem(big, medium, small);
```

```
 * boolean param_1 = obj.addCar(carType);
```

```
 */
```

Program-1(B)

// LeetCode 705 – Design HashSet

```
class MyHashSet {  
    private boolean[] mp;  
  
    public MyHashSet() {  
        mp = new boolean[1000001];  
        Arrays.fill(mp, false);  
    }  
  
    public void add(int key) {  
        mp[key] = true;  
    }  
  
    public void remove(int key) {  
        mp[key] = false;  
    }  
  
    public boolean contains(int key) {  
        return mp[key];  
    }  
}
```

Program-2

// 1396 – Design Underground System

```
class Passenger {
    int checkinTime;
    int checkoutTime;
    String checkinLocation;
    String checkoutLocation;

    public Passenger(String checkinLocation, int checkinTime) {
        this.checkinLocation = checkinLocation;
        this.checkinTime = checkinTime;
    }

    void checkout(String checkoutLocation, int checkoutTime) {
        this.checkoutLocation = checkoutLocation;
        this.checkoutTime = checkoutTime;
    }
}

class Route {
    String startStation;
    String endStation;
    int totalNumberOfTrips;
    long totalTimeSpentInTrips;

    public Route(String startStation, String endStation) {
```

```

        this.startStation = startStation;
        this.endStation = endStation;
    }

    double getAverageTime() {
        return (double) totalTimeSpentInTrips / totalNumberOfTrips;
    }

    void addTrip(int startTime, int endTime) {
        totalTimeSpentInTrips += endTime - startTime;
        totalNumberOfTrips++;
    }
}

class UndergroundSystem {

    Map<Integer, Passenger> currentPassengerMap;
    Map<String, Route> routeMap;

    public UndergroundSystem() {
        currentPassengerMap = new HashMap<>();
        routeMap = new HashMap<>();
    }

    public void checkIn(int id, String stationName, int t) {
        if (!currentPassengerMap.containsKey(id)) {
            Passenger passenger = new Passenger(stationName, t);
            currentPassengerMap.put(id, passenger);
        }
    }
}

```

```
}  
}
```

```
public void checkOut(int id, String stationName, int t) {  
    if (currentPassengerMap.containsKey(id)) {  
        Passenger passenger = currentPassengerMap.get(id);  
        passenger.checkout(stationName, t);  
        String routeKey = passenger.checkinLocation + "," + passenger.checkoutLocation;  
        Route route = routeMap.getOrDefault(routeKey, new Route(passenger.checkinLocation,  
passenger.checkoutLocation));  
        route.addTrip(passenger.checkinTime, passenger.checkoutTime);  
        routeMap.put(routeKey, route);  
        currentPassengerMap.remove(id);  
    }  
}
```

```
public double getAverageTime(String startStation, String endStation) {  
    return routeMap.get(startStation + "," + endStation).getAverageTime();  
}  
}
```

Program-3(A)

```
interface Shape {  
    double calculateArea();  
}  
  
class Circle implements Shape {  
    private double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public double getRadius() {  
        return radius;  
    }  
    @Override  
    public double calculateArea() {  
        return Math.PI * radius * radius;  
    }  
}  
  
class Rectangle implements Shape {  
    private double length;  
    private double width;  
  
    public Rectangle(double length, double width) {  
        this.length = length;  
        this.width = width;  
    }  
}
```

```
public double getLength() {  
    return length;  
}
```

```
public double getWidth() {  
    return width;  
}
```

```
@Override  
public double calculateArea() {  
    return length * width;  
}  
}
```

```
class Triangle implements Shape {  
    private double base;  
    private double height;  
  
    public Triangle(double base, double height) {  
        this.base = base;  
        this.height = height;  
    }  
  
    public double getBase() {  
        return base;  
    }  
}
```



```

    public double getHeight() {
        return height;
    }

    @Override
    public double calculateArea() {
        return 0.5 * base * height;
    }
}

public class ShapeAreaCalculator {
    public static void main(String[] args) {

        System.out.println("Shape Area Calculator");
        Circle circle = new Circle(5.0);
        Rectangle rectangle = new Rectangle(4.0, 6.0);
        Triangle triangle = new Triangle(3.0, 8.0);
        System.out.println("Circle:");
        System.out.println("Radius = " + circle.getRadius());
        System.out.println("Area of Circle = " + circle.calculateArea());
        System.out.println("Rectangle:");
        System.out.println("Length = " + rectangle.getLength() + ", Width = " +
rectangle.getWidth());
        System.out.println("Area of Rectangle = " + rectangle.calculateArea());

        System.out.println("Triangle:");
        System.out.println("Base = " + triangle.getBase() + ", Height = " + triangle.getHeight());
        System.out.println("Area of Triangle = " + triangle.calculateArea());
    }
}

```

Program- 3(B)

// PiCalculator.java

```
import java.util.Scanner;

public class PiCalculator {

    private int terms;

    private double computedPi;

    public final double PI = Math.PI;

    public PiCalculator(int terms) {

        this.terms = terms;

        System.out.println("Calculating Pi using Leibniz Series...");

        this.computedPi = calculateLeibniz(terms);

    }

    private double calculateLeibniz(int n) {

        double pi = 0.0;

        for (int i = 0; i < n; i++) {

            double term = 4.0 / (2 * i + 1);

            if (i % 2 != 0) {

                term = -term; // Alternate subtraction and addition

            }

            pi += term;

        }

        return pi;

    }

    public void getPiValue() {

        System.out.println("Public Method - Displaying Result:");

        System.out.println("Approximated value of Pi: " + computedPi);

    }

}
```

```

protected void displayPrecisionInfo() {
    System.out.println("Protected Method - Displaying Precision Info:");
    System.out.println("Precision used: " + terms + " terms");
    System.out.println("Series used: Leibniz Series (4/1-4/3+4/5-4/7+4/9...)");
}

protected double calculateArea(double radius) {
    return computedPi * radius * radius;
}

public void displayPrivateData() {
    System.out.println("Private Data - Accessed only within class:");
    System.out.println("Raw computed value (private): " + computedPi);
}

public static void main(String[] args) {
    System.out.println("Pi Calculator using Access Specifiers");
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter the number of terms for Pi approximation: ");
    int terms = scanner.nextInt();
    PiCalculator calc = new PiCalculator(terms);
    calc.getPiValue();
    calc.displayPrecisionInfo();
    calc.displayPrivateData();
    scanner.close();
}
}

```

Program-3(C)

// LeetCode 1232 – Check If It Is a Straight Line

```
class Solution {  
    public boolean checkStraightLine(int[][] coordinates) {  
        int x0 = coordinates[0][0], y0 = coordinates[0][1],  
            x1 = coordinates[1][0], y1 = coordinates[1][1];  
        int dx = x1 - x0, dy = y1 - y0;  
        for (int[] co : coordinates) {  
            int x = co[0], y = co[1];  
            if (dx * (y - y1) != dy * (x - x1))  
                return false;  
        }  
        return true;  
    }  
}
```

Program-4

//LeetCode 1114 – Print in Order

```
class Foo {  
    boolean one;  
    boolean two;  
    public Foo() {  
        one = false;  
        two = false;  
    }  
    public synchronized void first(Runnable printFirst) throws InterruptedException {  
        printFirst.run();  
        one = true;  
        notifyAll();  
    }  
    public synchronized void second(Runnable printSecond) throws InterruptedException {  
        while (!one)  
            wait();  
        printSecond.run();  
        two = true;  
        notifyAll();  
    }  
    public synchronized void third(Runnable printThird) throws InterruptedException {  
        while (!two)  
            wait();  
        printThird.run();  
    }  
}
```

Program-5(A)

// LeetCode 242 – Valid Anagram

```
class Solution {  
    public boolean isAnagram(String s, String t) {  
        if (s.length() != t.length()) {  
            return false;  
        }  
  
        int[] freq = new int[26];  
        for (int i = 0; i < s.length(); i++) {  
            freq[s.charAt(i) - 'a']++;  
            freq[t.charAt(i) - 'a']--;  
        }  
  
        for (int i = 0; i < freq.length; i++) {  
            if (freq[i] != 0) {  
                return false;  
            }  
        }  
  
        return true;  
    }  
}
```

Program-5(B)

//CharacterCounter.java

```
import java.util.Scanner;

public class CountCharacters {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a string: ");

        String input = scanner.nextLine();

        int vowels = 0;

        int consonants = 0;

        int digits = 0;

        int specialChars = 0;

        String lowerCaseInput = input.toLowerCase();

        for (int i = 0; i < lowerCaseInput.length(); i++) {

            char ch = lowerCaseInput.charAt(i);

            if (ch >= 'a' && ch <= 'z') {

                if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {

                    vowels++;

                } else {

                    consonants++;

                }

            }

            else if (ch >= '0' && ch <= '9') {

                digits++;

            }

            // c. Else if the character is not a whitespace, increment specialChars

            else if (ch != ' ' && ch != '\t' && ch != '\n') {
```

```
        specialChars++;
    }
}
System.out.println("Vowels: " + vowels);
System.out.println("Consonants: " + consonants);
System.out.println("Digits: " + digits);
System.out.println("Special Characters: " + specialChars);

scanner.close();
}
}
```


Program-6

```
// SimpleCalculator.java

import java.awt.*;
import java.awt.event.*;

public class SimpleCalculator extends Frame implements ActionListener {

    Label lblNum1, lblNum2, lblResult;

    TextField txtNum1, txtNum2;

    Button btnAdd, btnSub, btnMul, btnDiv, btnClear;

    public SimpleCalculator() {

        // Frame Settings

        setTitle("Simple Calculator");

        setSize(400, 250);

        setLayout(null); // Absolute layout for custom positioning

        lblNum1 = new Label("First Number:");
        lblNum1.setBounds(50, 50, 100, 20);
        add(lblNum1);

        txtNum1 = new TextField();
        txtNum1.setBounds(160, 50, 150, 20);
        add(txtNum1);

        lblNum2 = new Label("Second Number:");
        lblNum2.setBounds(50, 90, 100, 20);
        add(lblNum2);

        txtNum2 = new TextField();
        txtNum2.setBounds(160, 90, 150, 20);
        add(txtNum2);

        btnAdd = new Button("Add");
        btnAdd.setBounds(40, 140, 50, 30);
        add(btnAdd);
```

```
btnSub = new Button("Subtract");
btnSub.setBounds(100, 140, 60, 30);
add(btnSub);

btnMul = new Button("Multiply");
btnMul.setBounds(170, 140, 60, 30);
add(btnMul);

btnDiv = new Button("Divide");
btnDiv.setBounds(240, 140, 50, 30);
add(btnDiv);

btnClear = new Button("Clear");
btnClear.setBounds(300, 140, 50, 30);
add(btnClear);

// Label to display the result
lblResult = new Label("Result: ");
lblResult.setBounds(50, 190, 300, 20);
add(lblResult);

btnAdd.addActionListener(this);
btnSub.addActionListener(this);
btnMul.addActionListener(this);
btnDiv.addActionListener(this);
btnClear.addActionListener(this);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent we) {
        System.exit(0);
    }
});

setVisible(true);
```

```
}  
  
@Override  
public void actionPerformed(ActionEvent e) {  
    // Handle Clear Button  
    if (e.getSource() == btnClear) {  
        txtNum1.setText("");  
        txtNum2.setText("");  
        lblResult.setText("Result: ");  
        return;  
    }  
    try {  
        double num1 = Double.parseDouble(txtNum1.getText());  
        double num2 = Double.parseDouble(txtNum2.getText());  
        double result = 0;  
  
        if (e.getSource() == btnAdd) {  
            result = num1 + num2;  
            lblResult.setText("Result: " + result); // Step 7: Display result in a label  
        }  
        else if (e.getSource() == btnSub) {  
            result = num1 - num2;  
            lblResult.setText("Result: " + result);  
        }  
        else if (e.getSource() == btnMul) {  
            result = num1 * num2;  
            lblResult.setText("Result: " + result);  
        }  
        else if (e.getSource() == btnDiv) {
```

```
        if (num2 == 0) {
            lblResult.setText("Result: Cannot divide by zero");
        } else {
            result = num1 / num2;
            lblResult.setText("Result: " + result);
        }
    }
} catch (NumberFormatException ex) {
    lblResult.setText("Result: Invalid input! Please enter numbers.");
}
}

public static void main(String[] args) {
    new SimpleCalculator();
}
}
```

Program-7

```
// StudentFile.java

import java.io.*;
import java.util.Scanner;

public class StudentFile {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        String fileName = "students.txt";

        try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {

            System.out.println("Enter Details of 3 Students");

            for (int i = 1; i <= 3; i++) {

                System.out.println("Student " + i);

                System.out.print("Roll No: ");

                String rollNo = scanner.nextLine();

                System.out.print("Name: ");

                String name = scanner.nextLine();

                System.out.print("Marks: ");

                String marks = scanner.nextLine();

                writer.write(rollNo + ", " + name + ", " + marks);

                writer.newLine(); // Move to the next line

            }

            System.out.println("Student records saved successfully.\n");

        } catch (IOException e) {

            System.out.println("An error occurred while writing to the file.");

            e.printStackTrace();

        }

    }

}
```

```

System.out.println("Student Records");

try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    String line;
    // Read records line by line using readLine()
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("An error occurred while reading the file.");
    e.printStackTrace();
}

System.out.print("\nEnter Roll Number to Search: ");
String searchRollNo = scanner.nextLine();
boolean isFound = false;

try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] studentData = line.split(", ");
        if (studentData.length == 3 && studentData[0].equals(searchRollNo)) {
            System.out.println("Student Found");
            System.out.println("Roll No: " + studentData[0]);
            System.out.println("Name: " + studentData[1]);
            System.out.println("Marks: " + studentData[2]);
            isFound = true;
            break;
        }
    }
}

```

```
        if (!isFound) {  
            System.out.println("Student not found.");  
        }  
    } catch (IOException e) {  
        System.out.println("An error occurred while searching the file.");  
        e.printStackTrace();  
    }  
  
    scanner.close();  
}  
}
```

Program -7

```
// StudentFile.java
```

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
public class StudentFile {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        String fileName = "students.txt";
```

```
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
```

```
            System.out.println("Enter Details of 3 Students");
```

```
            for (int i = 1; i <= 3; i++) {
```

```
                System.out.println("Student " + i);
```

```
                System.out.print("Roll No: ");
```

```
                String rollNo = scanner.nextLine();
```

```
                System.out.print("Name: ");
```

```
                String name = scanner.nextLine();
```

```
                System.out.print("Marks: ");
```

```
                String marks = scanner.nextLine();
```

```
                writer.write(rollNo + ", " + name + ", " + marks);
```

```
                writer.newLine(); // Move to the next line
```

```
            }
```

```
            System.out.println("Student records saved successfully.\n");
```

```
        } catch (IOException e) {
```

```
            System.out.println("An error occurred while writing to the file.");
```

```
            e.printStackTrace();
```



```

}

System.out.println("Student Records");

try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    String line;
    // Read records line by line using readLine()
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("An error occurred while reading the file.");
    e.printStackTrace();
}

System.out.print("\nEnter Roll Number to Search: ");

String searchRollNo = scanner.nextLine();

boolean isFound = false;

try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] studentData = line.split(", ");
        if (studentData.length == 3 && studentData[0].equals(searchRollNo)) {
            // Step 11 (Procedure): Display the matching student's details
            System.out.println("Student Found");
            System.out.println("Roll No: " + studentData[0]);
            System.out.println("Name: " + studentData[1]);
            System.out.println("Marks: " + studentData[2]);
            isFound = true;
        }
    }
}

```

```
        break;
    }
}

if (!isFound) {
    System.out.println("Student not found.");
}

// Step 12 (Procedure): Close the BufferedReader (handled by try-with-resources)

} catch (IOException e) {
    System.out.println("An error occurred while searching the file.");
    e.printStackTrace();
}

scanner.close();
}
}
```

Program-8

```
// StudentRegistration.java

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class StudentRegistration extends JFrame implements ActionListener {

    JLabel title, lblUsn, lblName, lblBranch, lblGender, lblSkills;
    JTextField txtUsn, txtName;
    JComboBox<String> cbBranch;
    JRadioButton rbMale, rbFemale;
    ButtonGroup bgGender;
    JCheckBox chkJava, chkPython;
    JButton btnSubmit, btnClear;
    JTextArea txtAreaDetails;

    public StudentRegistration() {

        // Frame Settings

        setTitle("Student Registration Form");
        setSize(500, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null); // Using absolute positioning for custom layout

        // Title Label

        title = new JLabel("STUDENT REGISTRATION FORM");
        title.setFont(new Font("Arial", Font.BOLD, 18));
        title.setBounds(100, 10, 300, 30);
        add(title);


        // USN Label and TextField
```

```
lblUsn = new JLabel("USN:");
lblUsn.setBounds(50, 60, 100, 20);
add(lblUsn);

txtUsn = new JTextField();
txtUsn.setBounds(150, 60, 200, 20);
add(txtUsn);

// Name Label and TextField
lblName = new JLabel("Name:");
lblName.setBounds(50, 100, 100, 20);
add(lblName);

txtName = new JTextField();
txtName.setBounds(150, 100, 200, 20);
add(txtName);

// Branch Label and ComboBox
lblBranch = new JLabel("Branch:");
lblBranch.setBounds(50, 140, 100, 20);
add(lblBranch);

String[] branches = {"Computer Science", "Information Science", "Electronics",
"Mechanical"};

cbBranch = new JComboBox<>(branches);
cbBranch.setBounds(150, 140, 200, 20);
add(cbBranch);

lblGender = new JLabel("Gender:");
lblGender.setBounds(50, 180, 100, 20);
add(lblGender);

rbMale = new JRadioButton("Male");
rbMale.setBounds(150, 180, 80, 20);
rbFemale = new JRadioButton("Female");
```

```
rbFemale.setBounds(230, 180, 100, 20);
bgGender = new ButtonGroup();
bgGender.add(rbMale);
bgGender.add(rbFemale);
add(rbMale);
add(rbFemale);

// Skills Label and CheckBoxes
lblSkills = new JLabel("Skills:");
lblSkills.setBounds(50, 220, 100, 20);
add(lblSkills);

chkJava = new JCheckBox("Java");
chkJava.setBounds(150, 220, 80, 20);
chkPython = new JCheckBox("Python");
chkPython.setBounds(230, 220, 100, 20);
add(chkJava);
add(chkPython);

// Submit and Clear Buttons (Step 5: Register buttons)
btnSubmit = new JButton("Submit");
btnSubmit.setBounds(100, 270, 100, 30);
btnSubmit.addActionListener(this);
add(btnSubmit);

btnClear = new JButton("Clear");
btnClear.setBounds(250, 270, 100, 30);
btnClear.addActionListener(this);
add(btnClear);

// Text Area to display student details
txtAreaDetails = new JTextArea();
txtAreaDetails.setEditable(false);
```

```

JScrollPane scrollPane = new JScrollPane(txtAreaDetails);
scrollPane.setBounds(50, 320, 380, 150);
add(scrollPane);
setVisible(true);
}

@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnSubmit) {
        String usn = txtUsn.getText().trim();
        String name = txtName.getText().trim();
        String branch = (String) cbBranch.getSelectedItem();
        if (usn.isEmpty() || name.isEmpty() || (!rbMale.isSelected() && !rbFemale.isSelected()))
        {
            JOptionPane.showMessageDialog(this, "Please fill all mandatory fields (USN, Name,
and Gender).", "Validation Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        String gender = rbMale.isSelected() ? "Male" : "Female";
        StringBuilder skills = new StringBuilder();
        if (chkJava.isSelected()) skills.append("Java ");
        if (chkPython.isSelected()) skills.append("Python");
        if (skills.length() == 0) skills.append("None");
        txtAreaDetails.setText("--- Student Details ---\n");
        txtAreaDetails.append("USN: " + usn + "\n");
        txtAreaDetails.append("Name: " + name + "\n");
        txtAreaDetails.append("Branch: " + branch + "\n");
        txtAreaDetails.append("Gender: " + gender + "\n");
        txtAreaDetails.append("Skills: " + skills.toString() + "\n");
    }
}

```

```
else if (e.getSource() == btnClear) {  
    txtUsn.setText("");  
    txtName.setText("");  
    cbBranch.setSelectedIndex(0);  
    bgGender.clearSelection();  
    chkJava.setSelected(false);  
    chkPython.setSelected(false);  
    txtAreaDetails.setText("");  
}  
}  
public static void main(String[] args) {  
    new StudentRegistration();  
}  
}
```

Program-9

```
// StudentJDBC.java
```

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
public class StudentJDBC {
```

```
    static final String DB_URL = "jdbc:mysql://localhost:3306/college";
```

```
    static final String DB_USER = "root";    // Replace with your MySQL username
```

```
    static final String DB_PASSWORD = "password"; // Replace with your MySQL password
```

```
    public static void main(String[] args) {
```

```
        Connection conn = null;
```

```
        PreparedStatement pstmt = null;
```

```
        ResultSet rs = null;
```

```
        try {
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
```

```
            String insertQuery = "INSERT INTO student (rollno, name, department, marks)  
VALUES (?, ?, ?, ?)";
```

```
            pstmt = conn.prepareStatement(insertQuery);
```

```
            pstmt.setInt(1, 101);
```

```
            pstmt.setString(2, "Rahul");
```

```
            pstmt.setString(3, "CSE");
```

```
            pstmt.setInt(4, 85); // Initial marks before the update step
```

```
            pstmt.executeUpdate();
```



```
pstmt.setInt(1, 102);
pstmt.setString(2, "Sneha");
pstmt.setString(3, "ISE");
pstmt.setInt(4, 91);
pstmt.executeUpdate();
System.out.println("Records Inserted Successfully.");
String updateQuery = "UPDATE student SET marks = ? WHERE rollno = ?";
pstmt = conn.prepareStatement(updateQuery);
pstmt.setInt(1, 95); // Updating Rahul's marks to 95 as per sample output
pstmt.setInt(2, 101);
pstmt.executeUpdate();
System.out.println("Record Updated Successfully.");
String searchQuery = "SELECT * FROM student WHERE rollno = ?";
pstmt = conn.prepareStatement(searchQuery);
pstmt.setInt(1, 101);
rs = pstmt.executeQuery();
System.out.println("\nStudent Details");
if (rs.next()) {
    System.out.println("Roll No: " + rs.getInt("rollno"));
    System.out.println("Name : " + rs.getString("name"));
    System.out.println("Department: " + rs.getString("department"));
    System.out.println("Marks : " + rs.getInt("marks"));
}
rs.close(); // Close search ResultSet
String displayQuery = "SELECT * FROM student";
pstmt = conn.prepareStatement(displayQuery);
rs = pstmt.executeQuery();
System.out.println("\nStudent Records");
```

```

        System.out.println("Roll\tName\tDepartment\tMarks");
        while (rs.next()) {
            System.out.print(rs.getInt("rollno") + "\t");
            System.out.print(rs.getString("name") + "\t");
            System.out.print(rs.getString("department") + "\t\t");
            System.out.println(rs.getInt("marks"));
        } catch (ClassNotFoundException e) {
            System.out.println("MySQL JDBC Driver not found. Ensure the connector JAR is in
your classpath.");
            e.printStackTrace();
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                if (rs != null) rs.close();
                if (pstmt != null) pstmt.close();
                if (conn != null) conn.close();
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}

```

Program-10

```
//StudentRecord.java

import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;

class Student {
    int rollNo;
    String name;
    double percentage;
    public Student(int rollNo, String name, double percentage) {
        this.rollNo = rollNo;
        this.name = name;
        this.percentage = percentage;
    }
    @Override
    public String toString() {
        return rollNo + "\t" + name + "\t" + percentage;
    }
}

public class StudentRecord {
    public static void main(String[] args) {
        ArrayList<Student> studentList = new ArrayList<>();
        HashMap<Integer, Student> studentMap = new HashMap<>();
        Student s1 = new Student(101, "Rahul", 88.5);
        Student s2 = new Student(102, "Sneha", 91.2);
        Student s3 = new Student(103, "Kiran", 84.8);

        studentList.add(s1);
```

```
studentList.add(s2);
studentList.add(s3);
studentMap.put(s1.rollNo, s1);
studentMap.put(s2.rollNo, s2);
studentMap.put(s3.rollNo, s3);
System.out.println("Student Records (ArrayList)");
System.out.println("Roll\tName\tPercentage");
for (Student s : studentList) {
    System.out.println(s);
}
int searchRoll = 102;
System.out.println("\nSearching for Roll No " + searchRoll);
if (studentMap.containsKey(searchRoll)) {
    Student foundStudent = studentMap.get(searchRoll);
    System.out.println("Record Found");
    System.out.println("Roll No   : " + foundStudent.rollNo);
    System.out.println("Name      : " + foundStudent.name);
    System.out.println("Percentage: " + foundStudent.percentage);
} else {
    System.out.println("Record Not Found");
}
System.out.println("\nAfter Removing First Student");
studentList.remove(0);
System.out.println("Roll\tName\tPercentage");
for (Student s : studentList) {
    System.out.println(s);
}
System.out.println("\nStudent Records (HashMap)");
```

```
for (Map.Entry<Integer, Student> entry : studentMap.entrySet()) {  
    Student s = entry.getValue();  
    System.out.println(entry.getKey() + " -> " + s.name + " (" + s.percentage + "%");  
}  
}  
}
```

Program-10(A)

// ChessGame.java

```
import java.util.Scanner;
```

```
class Spot {
```

```
    private int x; // Row index (0-7)
```

```
    private int y; // Column index (0-7)
```

```
    private Piece piece;
```

```
    public Spot(int x, int y, Piece piece) {
```

```
        this.x = x;
```

```
        this.y = y;
```

```
        this.piece = piece;
```

```
    }
```

```
    public int getX() { return x; }
```

```
    public int getY() { return y; }
```

```
    public Piece getPiece() { return piece; }
```

```
    public void setPiece(Piece piece) { this.piece = piece; }
```

```
}
```

```
abstract class Piece {
```

```
    private boolean white;
```

```
    private boolean killed = false;
```

```
    public Piece(boolean white) {
```

```
        this.white = white;
```

```
    }
```

```
    public boolean isWhite() { return white; }
```

```
    public boolean isKilled() { return killed; }
```

```
    public void setKilled(boolean killed) { this.killed = killed; }
```

```

    public abstract boolean canMove(Board board, Spot start, Spot end);

    public abstract String getSymbol();
}

class Pawn extends Piece {
    public Pawn(boolean white) { super(white); }

    @Override
    public String getSymbol() { return isWhite() ? "P" : "p"; }

    @Override
    public boolean canMove(Board board, Spot start, Spot end) {
        if (end.getPiece() != null && end.getPiece().isWhite() == this.isWhite()) {
            return false;
        }

        int direction = isWhite() ? -1 : 1; // White moves up (towards row 0), Black moves down
        int startRow = isWhite() ? 6 : 1;
        int dx = end.getX() - start.getX();
        int dy = Math.abs(end.getY() - start.getY());

        if (dy == 0 && dx == direction && end.getPiece() == null) {
            return true;
        }

        if (dy == 0 && start.getX() == startRow && dx == 2 * direction && end.getPiece() ==
null) {
            int midX = start.getX() + direction;
            if (board.getSpot(midX, start.getY()).getPiece() == null) {
                return true;
            }
        }

        if (dy == 1 && dx == direction && end.getPiece() != null && end.getPiece().isWhite() !=
this.isWhite()) {

```

```

        return true;
    }

    return false;
}

class Rook extends Piece {
    public Rook(boolean white) { super(white); }
    @Override
    public String getSymbol() { return isWhite() ? "R" : "r"; }
    @Override
    public boolean canMove(Board board, Spot start, Spot end) {
        if (end.getPiece() != null && end.getPiece().isWhite() == this.isWhite()) return false;
        if (start.getX() != end.getX() && start.getY() != end.getY()) return false;
        int stepX = Integer.compare(end.getX(), start.getX());
        int stepY = Integer.compare(end.getY(), start.getY());
        int currX = start.getX() + stepX;
        int currY = start.getY() + stepY;
        while (currX != end.getX() || currY != end.getY()) {
            if (board.getSpot(currX, currY).getPiece() != null) return false;
            currX += stepX;
            currY += stepY;
        }
        return true;
    }
}

class Knight extends Piece {
    public Knight(boolean white) { super(white); }

```


@Override

```
public String getSymbol() { return isWhite() ? "N" : "n"; }
```

@Override

```
public boolean canMove(Board board, Spot start, Spot end) {  
    if (end.getPiece() != null && end.getPiece().isWhite() == this.isWhite()) return false;  
    int dx = Math.abs(start.getX() - end.getX());  
    int dy = Math.abs(start.getY() - end.getY());  
    return dx * dy == 2;  
}  
}
```

```
class Bishop extends Piece {
```

```
    public Bishop(boolean white) { super(white); }
```

@Override

```
public String getSymbol() { return isWhite() ? "B" : "b"; }
```

@Override

```
public boolean canMove(Board board, Spot start, Spot end) {  
    if (end.getPiece() != null && end.getPiece().isWhite() == this.isWhite()) return false;  
    int dx = Math.abs(start.getX() - end.getX());  
    int dy = Math.abs(start.getY() - end.getY());  
    if (dx != dy) return false;  
    int stepX = Integer.compare(end.getX(), start.getX());  
    int stepY = Integer.compare(end.getY(), start.getY());  
    int currX = start.getX() + stepX;  
    int currY = start.getY() + stepY;  
  
    while (currX != end.getX() || currY != end.getY()) {  
        if (board.getSpot(currX, currY).getPiece() != null) return false;  
        currX += stepX;
```

```

        currY += stepY;
    }
    return true;
}
}

class Queen extends Piece {
    public Queen(boolean white) { super(white); }

    @Override
    public String getSymbol() { return isWhite() ? "Q" : "q"; }

    @Override
    public boolean canMove(Board board, Spot start, Spot end) {
        if (end.getPiece() != null && end.getPiece().isWhite() == this.isWhite()) return false;
        int dx = Math.abs(start.getX() - end.getX());
        int dy = Math.abs(start.getY() - end.getY());
        if (dx != dy && start.getX() != end.getX() && start.getY() != end.getY()) return false;
        int stepX = Integer.compare(end.getX(), start.getX());
        int stepY = Integer.compare(end.getY(), start.getY());
        int currX = start.getX() + stepX;
        int currY = start.getY() + stepY;
        while (currX != end.getX() || currY != end.getY()) {
            if (board.getSpot(currX, currY).getPiece() != null) return false;
            currX += stepX;
            currY += stepY;
        }
        return true;
    }
}

class King extends Piece {

```

```

public King(boolean white) { super(white); }

@Override
public String getSymbol() { return isWhite() ? "K" : "k"; }

@Override
public boolean canMove(Board board, Spot start, Spot end) {
    if (end.getPiece() != null && end.getPiece().isWhite() == this.isWhite()) return false;
    int dx = Math.abs(start.getX() - end.getX());
    int dy = Math.abs(start.getY() - end.getY());
    return (dx <= 1 && dy <= 1);
}
}

class Board {
    private Spot[][] boxes = new Spot[8][8];

    public Board() {
        resetBoard();
    }

    public Spot getSpot(int x, int y) {
        if (x < 0 || x > 7 || y < 0 || y > 7) return null;
        return boxes[x][y];
    }

    public void resetBoard() {
        boxes[0][0] = new Spot(0, 0, new Rook(false));
        boxes[0][1] = new Spot(0, 1, new Knight(false));
        boxes[0][2] = new Spot(0, 2, new Bishop(false));
        boxes[0][3] = new Spot(0, 3, new Queen(false));
        boxes[0][4] = new Spot(0, 4, new King(false));
        boxes[0][5] = new Spot(0, 5, new Bishop(false));
        boxes[0][6] = new Spot(0, 6, new Knight(false));
    }
}

```

```

boxes[0][7] = new Spot(0, 7, new Rook(false));
for (int i = 0; i < 8; i++) {
    boxes[1][i] = new Spot(1, i, new Pawn(false));
}
for (int i = 2; i < 6; i++) {
    for (int j = 0; j < 8; j++) {
        boxes[i][j] = new Spot(i, j, null);
    }
}
for (int i = 0; i < 8; i++) {
    boxes[6][i] = new Spot(6, i, new Pawn(true));
}

boxes[7][0] = new Spot(7, 0, new Rook(true));
boxes[7][1] = new Spot(7, 1, new Knight(true));
boxes[7][2] = new Spot(7, 2, new Bishop(true));
boxes[7][3] = new Spot(7, 3, new Queen(true));
boxes[7][4] = new Spot(7, 4, new King(true));
boxes[7][5] = new Spot(7, 5, new Bishop(true));
boxes[7][6] = new Spot(7, 6, new Knight(true));
boxes[7][7] = new Spot(7, 7, new Rook(true));
}

public void displayBoard() {
    for (int i = 0; i < 8; i++) {
        System.out.print((8 - i) + " ");
        for (int j = 0; j < 8; j++) {
            Piece p = boxes[i][j].getPiece();
            if (p == null) {

```

```

        System.out.print(" ");
    } else {
        System.out.print(p.getSymbol() + " ");
    }
}
}
System.out.println();
}
System.out.println(" a b c d e f g h");
}
}

public class ChessGame {
    private Board board;
    private boolean whiteTurn;

    public ChessGame() {
        board = new Board();
        whiteTurn = true;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ChessGame game = new ChessGame();
        System.out.println("Chess Game (Console Version)");
        System.out.println("Initial Board Setup:");
        game.board.displayBoard();
        while (true) {
            System.out.println("\n" + (game.whiteTurn ? "White's turn." : "Black's turn.));
            System.out.print("Enter move (e.g., e2 e4) or 'exit' to quit: ");

```

```

String input = scanner.nextLine().trim();
if (input.equalsIgnoreCase("exit")) {
    System.out.println("Game terminated.");
    break;
}
String[] parts = input.split("\\s+");
if (parts.length != 2) {
    System.out.println("Invalid format! Please enter moves like 'e2 e4'.");
    continue;
}
String startStr = parts[0];
String endStr = parts[1];
Spot startSpot = parseSpot(game.board, startStr);
Spot endSpot = parseSpot(game.board, endStr);
if (startSpot == null || endSpot == null) {
    System.out.println("Invalid coordinates! Use columns a-h and rows 1-8.");
    continue;
}

Piece p = startSpot.getPiece();
if (p == null) {
    System.out.println("Error: No piece at start position " + startStr + ".");
    continue;
}
if (p.isWhite() != game.whiteTurn) {
    System.out.println("Error: It is " + (game.whiteTurn ? "White's" : "Black's") + "
turn!");
    continue;
}

```

```

        if (!p.canMove(game.board, startSpot, endSpot)) {
            System.out.println("Error: Invalid move for " + getPieceName(p) + " from " + startStr
+ " to " + endStr + ".");
            continue;
        }
        Piece destPiece = endSpot.getPiece();
        if (destPiece != null) {
            destPiece.setKilled(true);
        }
        endSpot.setPiece(p);
        startSpot.setPiece(null);
        System.out.println(getPieceName(p) + " moved from " + startStr + " to " + endStr + ".");
        game.board.displayBoard();
        if (destPiece instanceof King) {
            System.out.println("Checkmate!");
            System.out.println((game.whiteTurn ? "White" : "Black") + " wins the game.");
            break;
        }

        game.whiteTurn = !game.whiteTurn;
    }

    scanner.close();
}

private static Spot parseSpot(Board board, String coord) {
    if (coord.length() != 2) return null;
    char colChar = Character.toLowerCase(coord.charAt(0));
    char rowChar = coord.charAt(1);
    if (colChar < 'a' || colChar > 'h' || rowChar < '1' || rowChar > '8') {

```

```
        return null;
    }
    int col = colChar - 'a';
    int row = 8 - (rowChar - '0');
    return board.getSpot(row, col);
}
private static String getPieceName(Piece p) {
    if (p instanceof Pawn) return "Pawn";
    if (p instanceof Rook) return "Rook";
    if (p instanceof Knight) return "Knight";
    if (p instanceof Bishop) return "Bishop";
    if (p instanceof Queen) return "Queen";
    if (p instanceof King) return "King";
    return "Piece";
}
}
```


Program-10(B)

```
// EmployeeAnalytics.java

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.stream.Collectors;

class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    public Employee(int id, String name, String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    @Override
    public String toString() {
        return id + "\t" + name + "\t" + department + "\t" + salary;
    }
}
```

```

public class EmployeeAnalytics {

    public static void main(String[] args) {

        List<Employee> employees = Arrays.asList(
            new Employee(101, "Rahul", "CSE", 55000.0),
            new Employee(102, "Sneha", "ECE", 62000.0),
            new Employee(103, "Kiran", "CSE", 48000.0),
            new Employee(104, "Divya", "MECH", 51000.0),
            new Employee(105, "Arjun", "ECE", 70000.0)
        );

        System.out.println("---- All Employees ----");
        employees.forEach(e -> System.out.println(e));
        System.out.println("\n---- Salary Above 50000 (High to Low) ----");
        employees.stream()
            .filter(e -> e.getSalary() > 50000)
            .sorted((e1, e2) -> Double.compare(e2.getSalary(), e1.getSalary()))
            .forEach(e -> System.out.println(e.getName() + " -> " + e.getSalary()));
        System.out.println("\n---- Employee Names ----");
        List<String> employeeNames = employees.stream()
            .map(Employee::getName)
            .collect(Collectors.toList());

        System.out.println(employeeNames);
        System.out.println("\n---- Employees Grouped by Department ----");
        Map<String, List<String>> employeesByDept = employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.mapping(Employee::getName, Collectors.toList())
            ));

        employeesByDept.forEach((dept, names) -> System.out.println(dept + ": " + names));
    }
}

```

```

System.out.println("\n---- Average Salary per Department ----");
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
avgSalaryByDept.forEach((dept, avgSal) ->
    System.out.println(dept + ": " + String.format("%.2f", avgSal))
);
double totalSalary = employees.stream()
    .map(Employee::getSalary)
    .reduce(0.0, Double::sum);
System.out.println("\nTotal Salary Paid: " + String.format("%.2f", totalSalary));
long cseCount = employees.stream()
    .filter(e -> e.getDepartment().equalsIgnoreCase("CSE"))
    .count();
System.out.println("Number of CSE Employees : " + cseCount);

Optional<Employee> highestPaid = employees.stream()
    .max(Comparator.comparingDouble(Employee::getSalary));
highestPaid.ifPresent(e ->
    System.out.println("Highest Paid: " + e.getName() + " (" + e.getSalary() + ")")
);
}
}

```

Program-10(C)

// GenericDemo.java

```
class Box<T> {
    private T item;
    public void set(T item) {
        this.item = item;
    }
    public T get() {
        return item;
    }
    public void showType() {
        System.out.println("Type of stored item: " + item.getClass().getName());
    }
}

class Pair<K, V> {
    private K key;
    private V value;
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
    public void display() {
        System.out.println(key + " = " + value);
    }
}

public class GenericDemo {
    public static <T extends Comparable<T>> T findMax(T[] array) {
        if (array == null || array.length == 0) {
```

```

        return null;
    }
    T max = array[0];
    for (int i = 1; i < array.length; i++) {
        if (array[i].compareTo(max) > 0) {
            max = array[i];
        }
    }
    return max;
}

public static void main(String[] args) {
    Box<Integer> intBox = new Box<>();
    intBox.set(100);
    System.out.println("Integer Box Value: " + intBox.get());
    intBox.showType();
    Box<String> strBox = new Box<>();
    strBox.set("Hello Generics");
    System.out.println("String Box Value: " + strBox.get());
    strBox.showType();
    System.out.println("\n---- Key-Value Pairs ----");
    Pair<String, Integer> pair1 = new Pair<>("Rahul", 88);
    pair1.display();
    Pair<Integer, String> pair2 = new Pair<>(101, "CSE");
    pair2.display();
    Integer[] intArray = {45, 78, 89, 12};
    String[] strArray = {"Rahul", "Sneha", "Kiran"};
    Double[] doubleArray = {85.5, 75.0, 92.3};
    System.out.println("\nMaximum Number: " + findMax(intArray));
}

```

```

        System.out.println("Maximum (Alphabetical): " + findMax(strArray));
        System.out.println("Maximum Marks: " + findMax(doubleArray));
    }
}

```

Program-3(B)

```

// PiCalculator.java

import java.util.Scanner;

public class PiCalculator {
    private int terms;
    private double computedPi;
    public final double PI = Math.PI;
    public PiCalculator(int terms) {
        this.terms = terms;
        System.out.println("Calculating Pi using Leibniz Series...");
        this.computedPi = calculateLeibniz(terms);
    }
    private double calculateLeibniz(int n) {
        double pi = 0.0;
        for (int i = 0; i < n; i++) {
            double term = 4.0 / (2 * i + 1);
            if (i % 2 != 0) {
                term = -term; // Alternate subtraction and addition
            }
            pi += term;
        }
        return pi;
    }
    public void getPiValue() {

```

```

        System.out.println("Public Method - Displaying Result:");
        System.out.println("Approximated value of Pi: " + computedPi);
    }    protected void displayPrecisionInfo() {
        System.out.println("Protected Method - Displaying Precision Info:");
        System.out.println("Precision used: " + terms + " terms");
        System.out.println("Series used: Leibniz Series (4/1-4/3+4/5-4/7+4/9...)");
    }    protected double calculateArea(double radius) {
        return computedPi * radius * radius;
    }    public void displayPrivateData() {
        System.out.println("Private Data - Accessed only within class:");
        System.out.println("Raw computed value (private): " + computedPi);
    }

    public static void main(String[] args) {
        System.out.println("Pi Calculator using Access Specifiers");
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of terms for Pi approximation: ");
        int terms = scanner.nextInt();
        PiCalculator calc = new PiCalculator(terms);
        calc.getPiValue();
        calc.displayPrecisionInfo();
        calc.displayPrivateData();
        scanner.close();
    }
}

```